ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ІВАНА ФРАНКА
Факультет електроніки та комп'ютерних технологій

ЗВІТ
ДО ЛАБОРАТОРНОЇ РОБОТИ №2

«Структури даних та функціональна декомпозиція»

з курсу «Професійний Python»
Варіант №13 — «Система обліку комп'ютерної техніки»

Виконав: Капарник Дмитро Андрійович

Група: ФЕП-32

Перевірив: доцент Куньо Іван Михайлович

Дата: 18.10.2026

Львів — 2026

1. Мета роботи

Навчитися обирати структури даних, декомпозувати логіку на функції й зіставляти теоретичну складність із виміряним часом.

2. Індивідуальний варіант і постановка задачі

Варіант №13 — система обліку комп'ютерної техніки. Реалізувати унікальних виробників, групування за виробником або типом, Counter типів, індекс за номером і статистику вартості; використати comprehensions, *args, **kwargs, lambda, closure та decorator.

Наскрізний проєкт продовжує ЛР №1. Базові поля: інвентарний номер, тип, виробник, модель і вартість. Збережено пошук, фільтр виробника, середню/загальну вартість і визначення найдорожчого пристрою.

3. Структура проєкту та створені модулі

```
inventory-service/
  pyproject.toml, README.md, .gitignore
  src/computer_inventory/
    models.py
    services.py
    analytics.py
    main.py
  src/inventory_service/  # фінальний пакет запуску
  tests/
  scripts/
  docs/GITHUB.md
```

4. Структури даних і функціональна декомпозиція

Структура	Використання	Причина вибору
list[Device]	Початковий впорядкований список	Послідовне виведення й фільтрація
tuple	Пари (назва операції, час) в історії	Незмінний набір пов'язаних значень
set[str]	Унікальні виробники	Усунення повторів
dict[str, Device]	Індекс за номером	Швидкий точний пошук
Counter	Кількість пристроїв за типом	Підрахунок частот
defaultdict(list)	Групи пристроїв	Створення списку для нового ключа
deque(maxlen=10)	Історія виміряних операцій	Обмежений обсяг історії

```
def unique_manufacturers(devices: Iterable[Device]) -> set[str]:
    return {d.manufacturer for d in devices}

def build_index(devices: Iterable[Device]) -> dict[str, Device]:
```

```
    items = list(devices)
    index = {d.inventory_number.casefold(): d for d in items}
    if len(items) != len(index):
        raise ValueError("Duplicate inventory number")
    return index

def group_by(
    devices: Iterable[Device], field: str = "manufacturer"
) -> dict[str, list[Device]]:
    if field not in {"manufacturer", "device_type"}:
        raise ValueError("Unsupported grouping field")
    groups: defaultdict[str, list[Device]] = defaultdict(list)
    for device in devices:
        groups[str(getattr(device, field))].append(device)
    return dict(groups)

def count_types(devices: Iterable[Device]) -> Counter[str]:
    return Counter(d.device_type for d in devices)
```

5. Універсальні функції, closure та decorator

```
def merge(*groups: Iterable[Device]) -> list[Device]:
    return [device for group in groups for device in group]

def select(devices: Iterable[Device], **criteria: str) -> list[Device]:
    allowed = {"device_type", "manufacturer", "inventory_number", "model"}
    if not criteria.keys() <= allowed:
        raise ValueError("Unknown search field")
    return [
        d
        for d in devices
        if all(
            str(getattr(d, key)).casefold() == value.strip().casefold()
            for key, value in criteria.items()
        )
    ]

def make_cost_filter(minimum: Decimal) -> Callable[[Device], bool]:
    """The returned closure remembers the minimum cost."""
    return lambda device: device.cost >= minimum

def timed(label: str) -> Callable[[Callable[P, R]], Callable[P, R]]:
    def decorate(function: Callable[P, R]) -> Callable[P, R]:
        @wraps(function)
        def wrapped(*args: P.args, **kwargs: P.kwargs) -> R:
            start = perf_counter()
            try:
                return function(*args, **kwargs)
            finally:
                history.append((label, perf_counter() - start))

        return wrapped

    return decorate
```

```
@timed("summary")
def summarize(devices: Iterable[Device]) -> dict[str, object]:
    items = list(devices)
    best = most_expensive(items)
    return {
        "count": len(items),
        "total": str(total_cost(items)),
        "average": str(average_cost(items)),
        "most_expensive": best.inventory_number if best else None,
        "manufacturers": sorted(unique_manufacturers(items)),
        "types": dict(count_types(items)),
    }
```

merge приймає кілька наборів через *args, select — іменовані критерії через **kwargs. make_cost_filter повертає lambda, яка пам'ятає minimum. Параметризований timed зберігає назву й час операції у deque; wraps зберігає метадані функції. Грошові підсумки делегуються перевіреним функціям ЛР1.

6. Результати виконання

```
{
  "count": 4,
  "total": "92700.00",
  "average": "23175.00",
  "most_expensive": "INV-003",
  "manufacturers": [
    "Dell",
    "HP",
    "Lenovo"
  ],
  "types": {
    "Ноутбук": 2,
    "Монітор": 1,
    "Принтер": 1
  }
}
Dell: ['INV-002', 'INV-003']
Groups: {'Lenovo': 1, 'Dell': 2, 'HP': 1}
Counter: Counter({'Ноутбук': 2, 'Монітор': 1, 'Принтер': 1})
Closure: ['INV-001', 'INV-003']
```

7. Складність та експеримент list проти dict

Операція	**Складність часу**	**Додаткова пам'ять**
Лінійний точний пошук	O(n)	O(1)
Побудова dict-index	O(n) у середньому	O(n)
Пошук за готовим ключем	O(1) у середньому	O(1) на запит
Групування, Counter	O(n) у середньому	O(n) / O(k)

Операція	**Складність часу**	**Додаткова пам’ять**
Сортування за ціною	O(n log n)	O(n) для нового списку

Записів	**list, мкс/запит**	**dict, мкс/запит**	**Побудова індексу, мс**
1000	21.108	0.056	0.072
10000	214.366	0.057	1.007
100000	2394.588	0.057	17.326

Пошук виконувався для останнього номера. Наведено медіану п’яти серій: 50 лінійних запитів та 50 000 запитів до готового індексу в кожній серії. Час побудови індексу виміряно окремо. Для одного запиту його вартість може перевищувати виграш; індекс особливо корисний для повторних пошуків. Вимірювання виконано на Python 3.12.14 у Windows.

Код і історія лабораторних: https://github.com/dimakaparnyk/professional-python-labs. Для відповідного етапу використовується тег lab-02, фінальна інтегрована реалізація — main (перевірений CI commit 2e3fb3d).

8. Відповіді на контрольні запитання

1. Що таке list?

Впорядкована змінювана послідовність об’єктів із доступом за індексом.

2. Чим list відрізняється від tuple?

list можна змінювати, а tuple зберігає незмінний набір посилань на елементи.

3. Чому tuple називається immutable?

Після створення tuple не можна замінити, додати або видалити його елемент; вкладений змінюваний об’єкт може змінюватися.

4. Для чого використовується set?

Для зберігання унікальних значень і операцій об’єднання, перетину та різниці.

5. Яка властивість елементів set є важливою?

Елементи мають бути hashable: їхній хеш і порівняння мають залишатися узгодженими.

6. Що таке dict?

Змінюване відображення унікальних ключів у значення.

7. Що таке key-value structure?

Структура, де кожному ключу відповідає певне значення, наприклад інвентарному номеру — пристрій.

8. Чому пошук key у dict зазвичай швидкий?

Хеш-таблиця звужує пошук до комірки за хешем ключа; середня складність доступу — O(1).

9. Що таке hashability?

Можливість обчислити стабільний хеш об’єкта, узгоджений із його рівністю.

10. Чому list не може бути key у dict?

list змінюваний і не має стабільного хешу, тому є unhashable.

11. Для чого використовується Counter?

Для підрахунку кількості входжень; у проєкті — пристроїв кожного типу.

12. Для чого використовується defaultdict?

Для автоматичного створення початкового значення за відсутнім ключем; зручно групувати пристрої у списки.

13. Для чого використовується deque?

Для ефективних операцій на обох кінцях черги; maxlen обмежує історію операцій.

14. Що таке list comprehension?

Стислий вираз побудови списку з ітерацією та необов’язковою умовою.

15. Що таке set comprehension?

Вираз побудови множини з автоматичним усуненням повторів.

16. Що таке dict comprehension?

Вираз побудови словника з парами ключ: значення.

17. Що таке generator expression?

Лінивий вираз, що створює ітератор та обчислює елементи під час споживання.

18. Що означає O(1)?

Час або обсяг пам’яті асимптотично не залежить від кількості елементів.

19. Що означає O(n)?

Витрати зростають приблизно пропорційно кількості елементів.

20. Що означає O(n log n)?

Витрати зростають як n помножене на логарифм n; типово для порівняльного сортування.

21. Що означає $O(n^2)$?

Квадратичне зростання витрат, наприклад перебір усіх пар у двох вкладених циклах.

22. Яка приблизна складність пошуку у list?

O(n) у найгіршому випадку; перший збіг може бути знайдений раніше.

23. Яка середня складність пошуку у set?

У середньому O(1), у патологічному випадку колізій — O(n).

24. Яка середня складність пошуку key у dict?

У середньому O(1); побудова індексу для n елементів потребує O(n).

25. Чому правильна структура даних може прискорити програму?

Вона зменшує потрібну кількість операцій: індекс замінює повторний лінійний перебір.

26. Що таке функціональна декомпозиція?

Поділ задачі на невеликі функції з чіткими параметрами та результатами.

27. Які переваги малих functions?

Їх простіше перевіряти, повторно використовувати й змінювати незалежно.

28. Що таке positional argument?

Аргумент, значення якого зіставляється з параметром за позицією у виклику.

29. Що таке keyword argument?

Аргумент, переданий явно за назвою параметра.

30. Для чого використовується *args?

Для приймання довільної кількості позиційних аргументів у tuple; merge об’єднує кілька наборів.

31. Для чого використовується **kwargs?

Для приймання іменованих аргументів у dict; select приймає критерії фільтрації.

32. Що таке lambda function?

Невелика анонімна функція з одним виразом.

33. Де доцільно використовувати lambda?

Як короткий ключ сортування, предикат фільтра або інший простий callback.

34. Чому складну business logic не варто записувати в lambda?

Один вираз погіршує читабельність складних перевірок і ускладнює пояснення та тестування.

35. Що таке closure?

Функція, яка зберігає доступ до змінних зовнішньої функції після завершення її виклику.

36. Які variables може пам'ятати closure?

Змінні охоплювальної лексичної області, які використовує вкладена функція; у make_cost_filter це minimum.

37. Що таке decorator?

Обгортка, яка отримує функцію та повертає функцію зі зміненою або доданою поведінкою.

38. Для чого застосовуються decorators?

Для вимірювання часу, журналювання, кешування й перевірки доступу без дублювання коду.

39. Для чого використовується functools.wraps?

Для збереження імені, документації та посилання __wrapped__ вихідної функції.

40. Чому потрібно оцінювати складність алгоритму?

Щоб оцінювати масштабованість, вибирати структури даних і пояснювати результати вимірювань.

9. Висновок

Реалізовано модуль аналітики обладнання з усіма заданими структурами даних. Експеримент підтвердив лінійне зростання часу перебору й майже сталий час запиту до готового dict-index. Функції та бізнес-логіка відокремлені від інтерфейсу; підготовлено основу для потокової обробки.